

Bilingual English–Romanian Text Summarization

Buian Dragos

June 2, 2026

1 Problem statement

Abstractive text summarization produces a short summary that captures the main ideas of a longer document. In this project we address **bilingual summarization** for **English (EN)** and **Romanian (RO)**: given a source article, the system must generate a concise summary in the same language.

The task is challenging because:

- summaries must be fluent and informative (semantic quality);
- generated text should remain faithful to the source (factual consistency);
- a single multilingual model must handle both languages and multiple dataset formats.

2 Proposed solution

2.1 Theoretical aspects

We treat summarization as **sequence-to-sequence (seq2seq) learning**. Let $x = (x_1, \dots, x_n)$ be the tokenized source document and $y = (y_1, \dots, y_m)$ the reference summary. The model learns parameters θ that maximize the conditional likelihood:

$$\mathcal{L}(\theta) = \sum_{(x,y) \in \mathcal{D}} \log P_{\theta}(y \mid x) = \sum_{(x,y) \in \mathcal{D}} \sum_{t=1}^m \log P_{\theta}(y_t \mid y_{<t}, x).$$

We fine-tune **mT5** (google/mt5-small), a multilingual T5 encoder–decoder, on a mixed EN+RO training set. At inference time we use **beam search** (beam width $B = 4$) to approximate

$$\hat{y} = \arg \max_y P_{\theta}(y \mid x).$$

Semantic quality is measured with **BERTScore** (contextual embedding similarity between prediction and reference). **Factual consistency** is

measured with **SummaC**, which scores whether the summary is supported by the source.

2.2 Data sets used in the application

All corpora are loaded via Hugging Face **datasets** and normalized to a common schema: **id**, **language**, **dataset**, **source**, **reference**. If a corpus lacks **validation** or **test**, splits are created automatically (e.g. 80/10/10 from train).

Table 1: Corpora used in the pipeline.

Dataset	Hugging Face id and field mapping
EN CNN/DailyMail	<code>cnn_dailymail</code> article $\rightarrow$ source, highlights $\rightarrow$ reference
EN XSum	<code>xsum</code> document $\rightarrow$ source, summary $\rightarrow$ reference
RO ReaderBench	<code>readerbench/ro-text-summarization</code> Content $\rightarrow$ source, Summary $\rightarrow$ reference
RO MassiveSumm	<code>MaLA-LM/MassiveSumm_long</code> (language <code>ron</code>) text $\rightarrow$ source, summary $\rightarrow$ reference

Training, validation, and test examples from all selected datasets are **concatenated and shuffled** (with optional per-dataset caps for smoke runs).

2.3 Application

The application is a command-line **pipeline** with stages: prepare $\rightarrow$ train $\rightarrow$ infer $\rightarrow$ eval (optional smoke test).

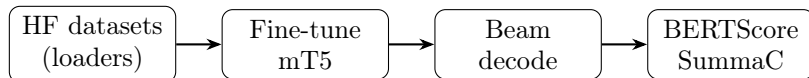


Figure 1: End-to-end workflow (`python -m src.pipeline`).

Outputs are written under `practical/outputs/`: checkpoints, JSONL predictions, and JSON/CSV evaluation reports.

3 Implementation — details

3.1 Project layout

`practical/`

```

src/
  config.py          # paths, model names, lengths, seed
  pipeline.py        # orchestrates stages
  train.py           # Seq2SeqTrainer fine-tuning
  infer.py           # batch generation
  data/loaders.py    # dataset load + normalize + merge
  eval/
    bertscore_eval.py
    summac_eval.py
outputs/             # checkpoints, predictions, reports

```

3.2 Libraries and main APIs

- **PyTorch / Hugging Face Transformers** — `AutoModelForSeq2SeqLM`, `AutoTokenizer`, `Seq2SeqTrainer`, `Seq2SeqTrainingArguments`, `DataCollatorForSeq2Seq`.
- **Hugging Face Datasets** — `load_dataset`, `concatenate_datasets`, `split` helpers in `src/data/loaders.py`.
- **bert-score** — multilingual `BERTScore` (`xlm-roberta-large` by default).
- **SummaC** — factual consistency scoring on (source, prediction) pairs.
- **accelerate** — training backend used by Transformers.

3.3 Key functions and entry points

- `load_all_datasets`, `build_combined_split` — load EN/RO corpora and build mixed splits.
- `tokenize_dataset` — encode sources and summary labels for training.
- `src.train` — fine-tune and save checkpoint to `outputs/checkpoints/mt5-small-en-ro`.
- `src.infer` — write `test_predictions.jsonl`.
- `src.eval.bertscore_eval`, `src.eval.summac_eval` — produce report files.
- `src.pipeline` — run selected stages from one CLI.

3.4 Default hyperparameters

Setting	Value
Base model	<code>google/mt5-small</code>
Max source / target length	512 / 128
Learning rate	5×10^{-5}
Beam size (inference)	4
Random seed	42

4 Experiments and results

4.1 Experimental setup

All runs were executed on a workstation with an **NVIDIA GeForce GTX 1070** (8 GB VRAM), using the conda environment defined in `environment.yml`. We use `google/mt5-small` rather than `mt5-base` to fit GPU memory and reduce training time. Table 2 summarizes the pilot experiment reported below.

Table 2: Pilot training and evaluation configuration.

Setting	Value
Model	<code>google/mt5-small</code>
EN datasets	CNN/DailyMail, XSum
RO datasets	ReaderBench, MassiveSumm (<code>ron</code>)
Cap per dataset	2 000 (train / val / test)
Epochs	1
Batch size (train / eval)	2 / 2
Max source / target length	512 / 128
Optimizer	AdamW, lr = 5×10^{-5} , warmup 0.05
Inference	beam search ($B = 4$)
Metrics	BERTScore (<code>xlm-roberta-large</code>), SummaC (<code>vitc</code>)
Checkpoint	<code>outputs/checkpoints/mt5-small-en-ro</code>

Training consumed $\sim 6\,000$ mixed examples ($\sim 3\,000$ optimization steps) and completed in ~ 18 minutes on the target GPU. The final training loss was 5.05; validation loss after one epoch was 2.86. Inference on the combined test split produced **6 000** predictions: 2 000 per corpus (CNN/DailyMail, XSum, ReaderBench).

4.2 Semantic quality (BERTScore)

BERTScore F1 was computed on all 6 000 test predictions against the reference summaries. Table 3 reports mean F1 by language and dataset.

Scores are high and stable across languages (~ 0.87 – 0.88), indicating that the generated summaries are semantically close to the references under this

Table 3: BERTScore F1 on the test split ($n = 6\,000$).

Lang.	Dataset	n	F1
EN	CNN/DailyMail	2 000	0.878
EN	XSum	2 000	0.874
RO	ReaderBench	2 000	0.875
Macro-average			0.875

shallow fine-tuning regime. This is expected partly because mT5-small is pre-trained and the evaluation set overlaps the fine-tuning distribution; stronger gains would require longer training or full-corpus fine-tuning.

4.3 Factual consistency (SummaC)

SummaC is substantially slower than BERTScore on long news articles, so we first evaluated a **random 200-example subset** of the prediction file (`summac_report_200.json`). Table 4 lists mean SummaC scores and the fraction of summaries marked factual at threshold 0.5.

Table 4: SummaC on a 200-example test subset (threshold = 0.5).

Lang.	Dataset	n	Mean score
EN	CNN/DailyMail	58	−0.107
EN	XSum	66	−0.095
RO	ReaderBench	76	0.282

On this subset, **25%** of summaries exceeded the factuality threshold (score ≥ 0.5). English news domains (CNN/DailyMail, XSum) received negative mean SummaC scores, while Romanian ReaderBench summaries scored higher on average. This pattern suggests that semantic overlap with references (BERTScore) does not imply source faithfulness: the model can produce fluent text that is poorly supported by the source, especially on abstractive English headlines.

4.4 Discussion and next steps

The pilot run validates the full pipeline (load $\rightarrow$ train $\rightarrow$ infer $\rightarrow$ eval) on bilingual data with practical resource constraints. Limitations of this experiment include: (i) only one epoch on capped subsets, (ii) a smaller encoder (`mt5-small`), and (iii) partial SummaC evaluation due to runtime. Promising follow-ups are full-corpus fine-tuning, mixed-precision training on GPU, faster greedy decoding for development, and completing SummaC on the full 6 000-example test set.